

系统编程

基于华为TaiShan服务器/openEuler OS 的实践

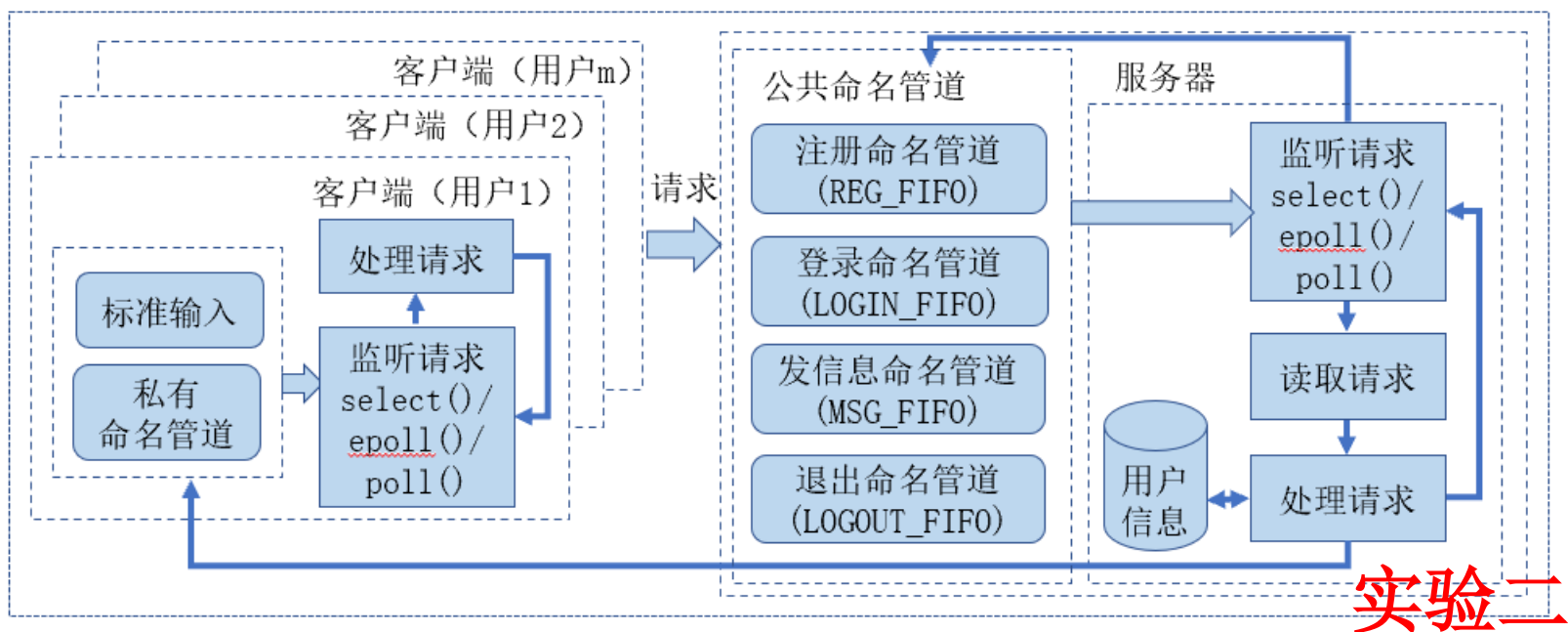
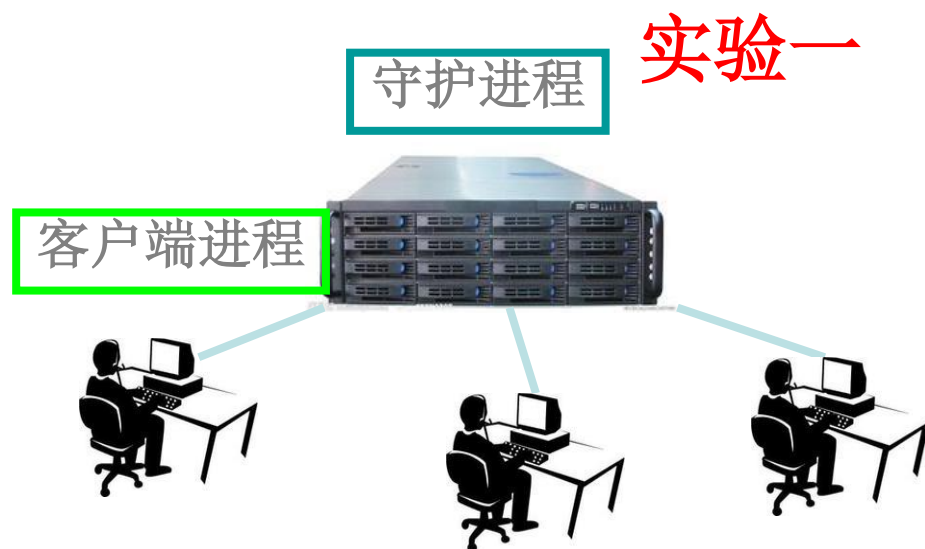
第三讲：多线程编程（上）－线程控制

① 进程 **vs.** 线程

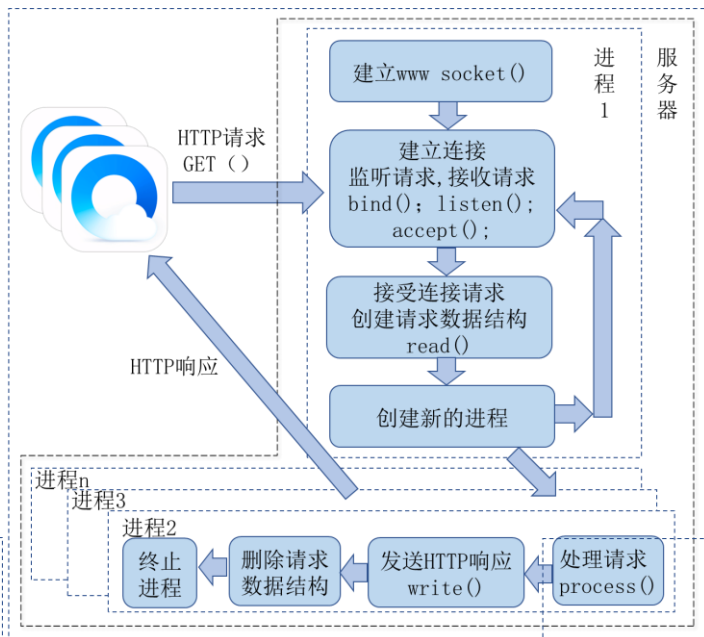
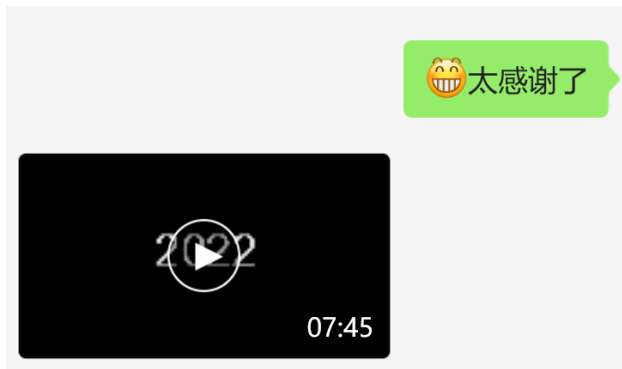
② 系统调用

回顾

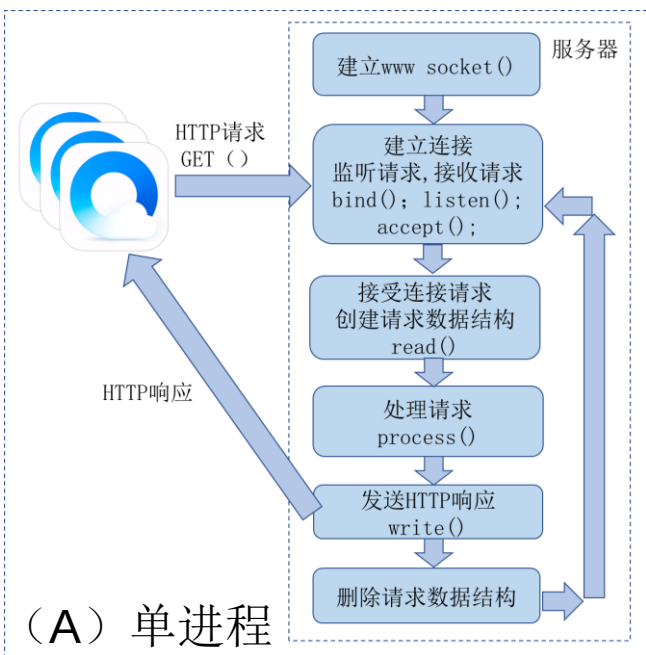
- 第一讲：进程控制
- 第二讲：进程间通信



更常见的例子- Web 服务器 (B) 多进程



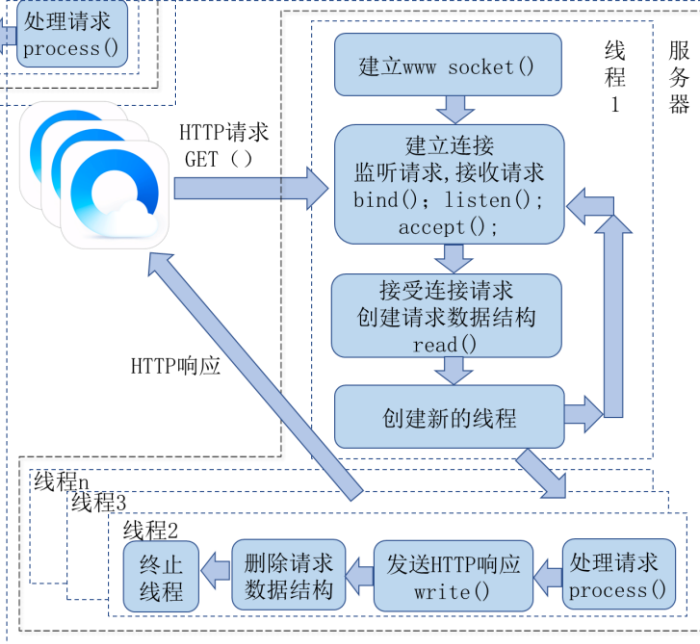
(C) 多线程



问题:

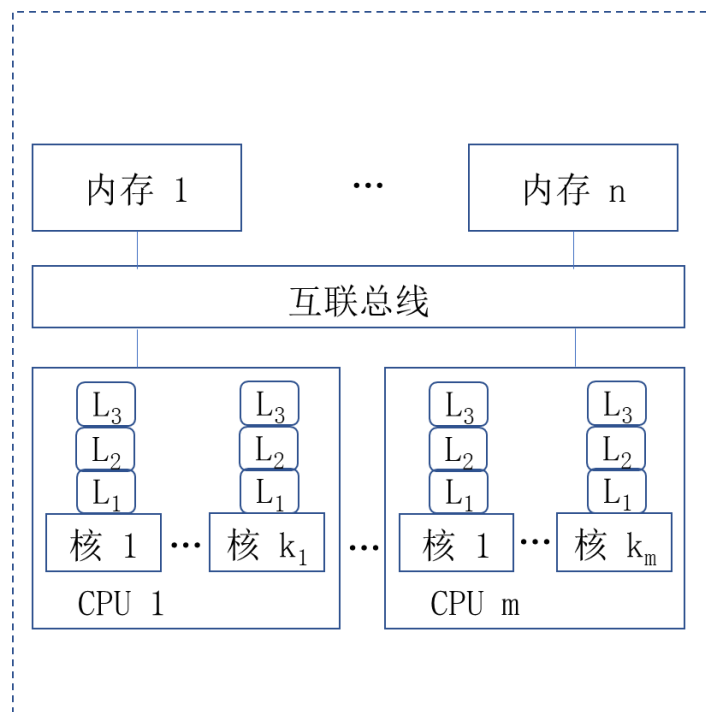
1) 选哪种实现方式?

2) 为什么?

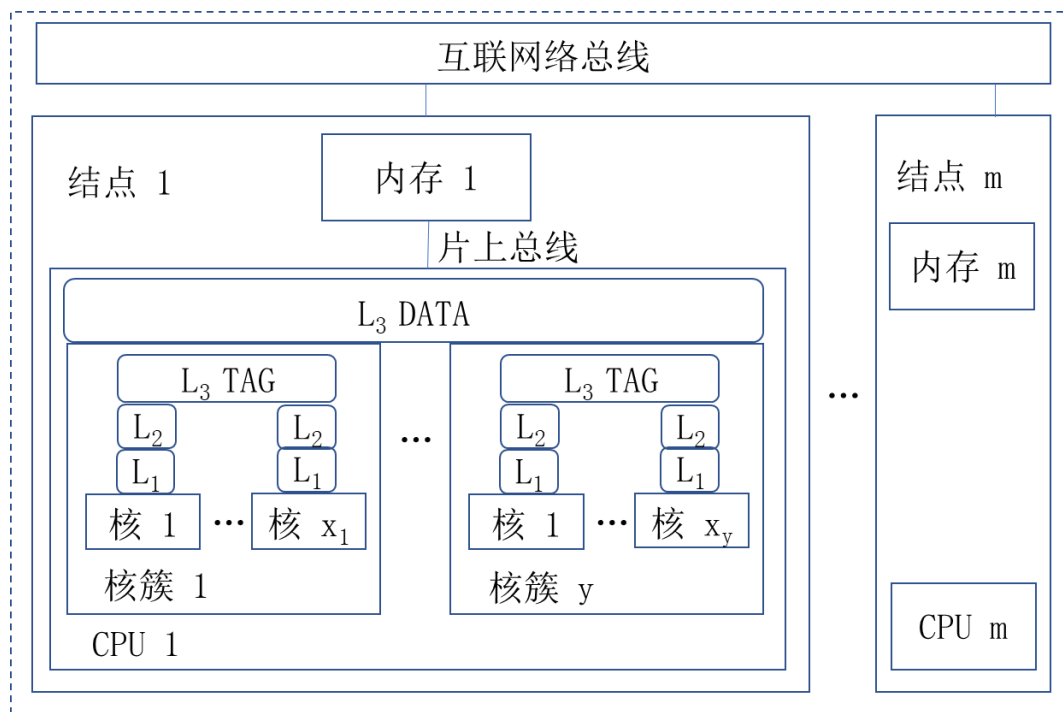


线程控制 – ① 进程 vs. 线程 – 多核处理器架构

- (1) 共享内存体系结构 (share memory architecture, SMA)
- (2) 分布式内存体系结构 (distributed memory architecture, DMA)



(a) 统一/均匀访存模型
(uniform memory access, UMA)



(b) 非统一/均匀访存模型
(nonuniform memory access, NUMA)

线程控制 – ① 进程 vs. 线程 – 多核处理器架构

看看咱们的2号Taishan服务器的处理器架构

```
[root@taishan02 ~]# lscpu
架构: aarch64
CPU 运行模式: 64-bit
字节序: Little Endian
CPU: 96
在线 CPU 列表: 0-95
每个核的线程数: 1
每个座的核数: 48
座: 2
NUMA 节点: 4
厂商 ID: HiSilicon
型号: 0
型号名称: Kunpeng-920
步进: 0x1
CPU 最大 MHz: 2600.0000
CPU 最小 MHz: 200.0000
BogoMIPS: 200.00
L1d 缓存: 6 MiB
L1i 缓存: 6 MiB
L2 缓存: 48 MiB
L3 缓存: 192 MiB
NUMA 节点0 CPU: 0-23
NUMA 节点1 CPU: 24-47
NUMA 节点2 CPU: 48-71
NUMA 节点3 CPU: 72-95
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected
Vulnerability Spec store bypass: Vulnerable
Vulnerability Spectre v1: Mitigation; __user pointer sanitization
Vulnerability Spectre v2: Not affected
Vulnerability Tsx async abort: Not affected
标记: fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt fcma dcpop
asimddp asimdghm
```

线程控制 – ① 进程 vs. 线程 – 多核处理器架构

```
[root@taishan02-vm-10 ~]$ lscpu
架构: aarch64
CPU 运行模式: 64-bit
字节序: Little Endian
CPU: 4
在线 CPU 列表: 0-3
每个核的线程数: 1
每个座的核数: 2
座: 2
NUMA 节点: 1
厂商 ID: HiSilicon
型号: 0
型号名称: Kunpeng-920
步进: 0x1
CPU 最大 MHz: 2600.0000
CPU 最小 MHz: 2600.0000
BogoMIPS: 200.00
L1d 缓存: 256 KiB
L1i 缓存: 256 KiB
L2 缓存: 2 MiB
L3 缓存: 64 MiB
NUMA 节点0 CPU: 0-3
Vulnerability Itlb multihit: Not affected
Vulnerability L1tf: Not affected
Vulnerability Mds: Not affected
Vulnerability Meltdown: Not affected
Vulnerability Spec store bypass: Vulnerable
Vulnerability Spectre v1: Mitigation; __user pointer sanitization
Vulnerability Spectre v2: Not affected
Vulnerability Tsx async abort: Not affected
标记: fp asimd evtstrm aes pmull sha1 sha2 crc32 atomic
      s fphp asimdhp cpuid asimdrdm jscvt fcma dcpop as
      imddp asimdfhm
```

咱们虚拟机的
处理器架构

线程控制 – ① 进程 vs. 线程 – 多核处理器架构

```
[root@taishan02-vm-10 ~]$ cat /proc/cpuinfo | more
processor       : 0
BogoMIPS      : 200.00
Features      : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp
cpuid asimdrrdm jscvt fcma dcpop asimddp asimdfhm
CPU implementer : 0x48
CPU architecture: 8
CPU variant    : 0x1
CPU part      : 0xd01
CPU revision   : 0

processor       : 1
BogoMIPS      : 200.00
Features      : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp
cpuid asimdrrdm jscvt fcma dcpop asimddp asimdfhm
CPU implementer : 0x48
CPU architecture: 8
CPU variant    : 0x1
CPU part      : 0xd01
CPU revision   : 0

processor       : 2
BogoMIPS      : 200.00
Features      : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp
cpuid asimdrrdm jscvt fcma dcpop asimddp asimdfhm
CPU implementer : 0x48
CPU architecture: 8
CPU variant    : 0x1
CPU part      : 0xd01
CPU revision   : 0

processor       : 3
BogoMIPS      : 200.00
Features      : fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp
cpuid asimdrrdm jscvt fcma dcpop asimddp asimdfhm
CPU implementer : 0x48
CPU architecture: 8
CPU variant    : 0x1
CPU part      : 0xd01
CPU revision   : 0
```

咱们虚拟机的
处理器架构

线程控制 – ① 进程 vs. 线程 – 进程模型和线程模型

运行环境

进程

线程

(a) 单进程单线程

运行环境

进程

线程 1

...

线程 k_1

(b) 单进程多线程

运行环境

进程 1

线程

...

进程 n

线程

(c) 多进程单线程

运行环境

进程 1

线程 1

线程 k_1

...

进程 n

线程 1

线程 k_n

(d) 多进程多线程

进程1
(任务一)

内核态
栈

PCB

用户态
栈

用户地址
空间

进程2
(任务二)

内核态
栈

PCB

用户态
栈

用户地址
空间

进程3
(任务三)

内核态
栈

PCB

用户态
栈

用户地址
空间

(e) 单线程进程

线程1
(任务一)

TCB

内核态
栈

用户态
栈

线程2
(任务二)

TCB

内核态
栈

用户态
栈

线程3
(任务三)

TCB

内核态
栈

用户态
栈

进程 1

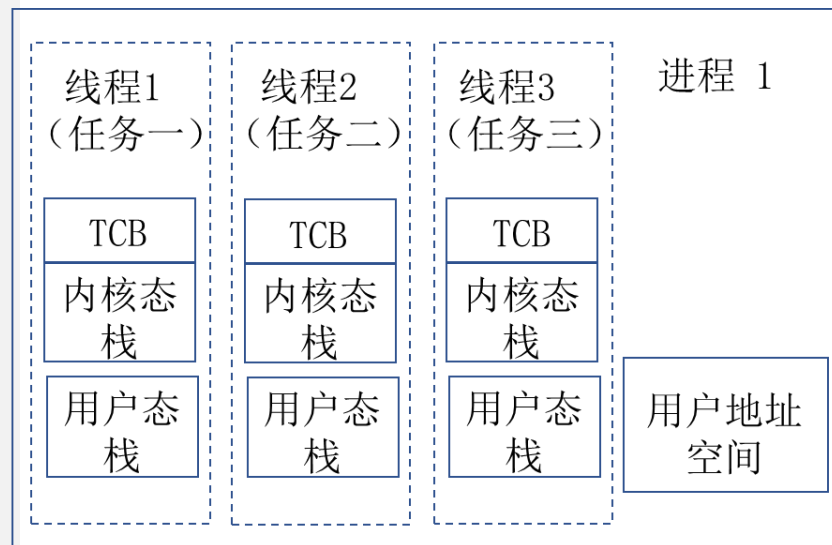
用户地址
空间

(f) 多线程进程

线程控制 – ① 进程 vs. 线程 – 进程模型和线程模型

■ 线程的优点

- 生成时间短
- 终止时间短
- 线程间通信时间短
 - ◆ 共享地址空间
 - ◆ 不用切换内核态传递消息
- 线程间切换时间短
 - ◆ 资源共享
 - ◆ 不用恢复用户地址空间



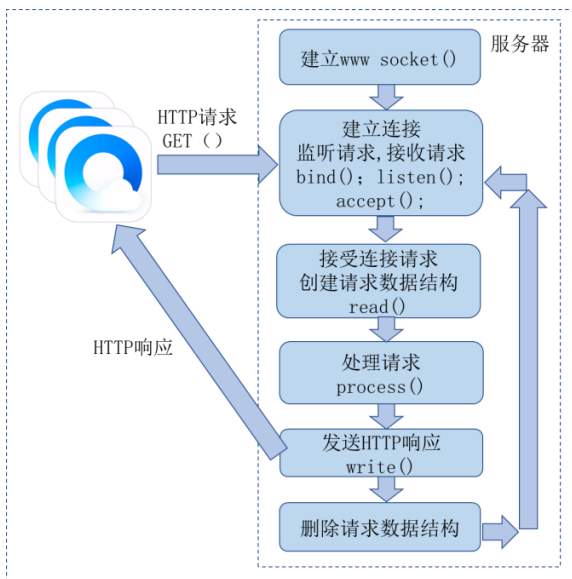
■ 进程

- 资源分配单元
- 处理机分配单元

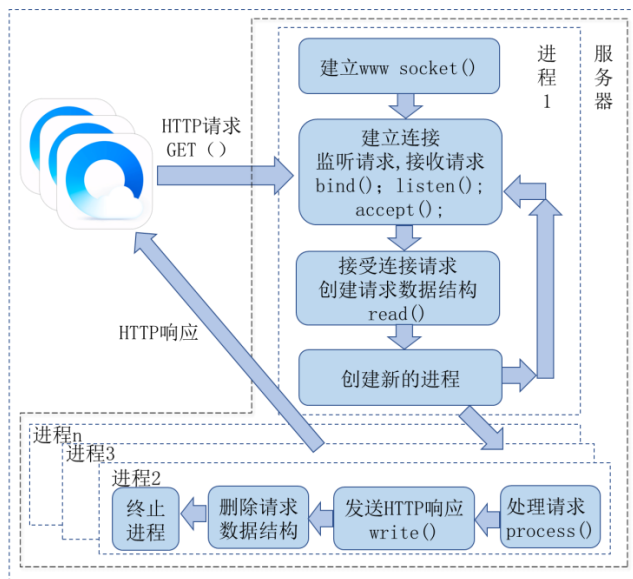
■ 线程

- 处理机分配单元

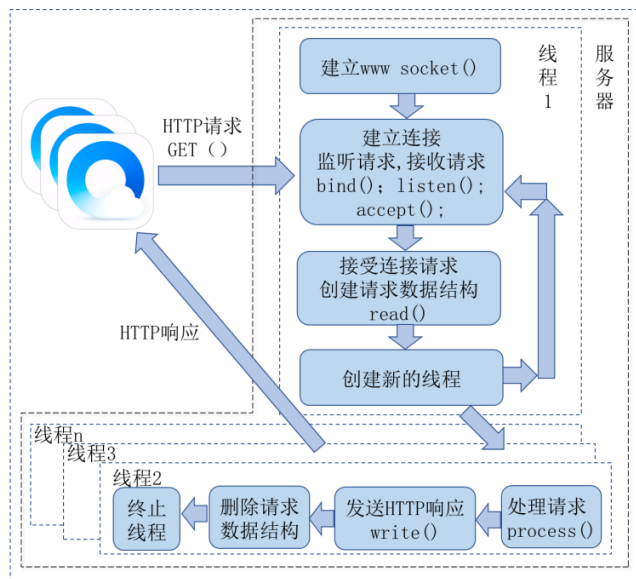
小结



(A) 单进程



(B) 多进程



(C) 多线程

问题:

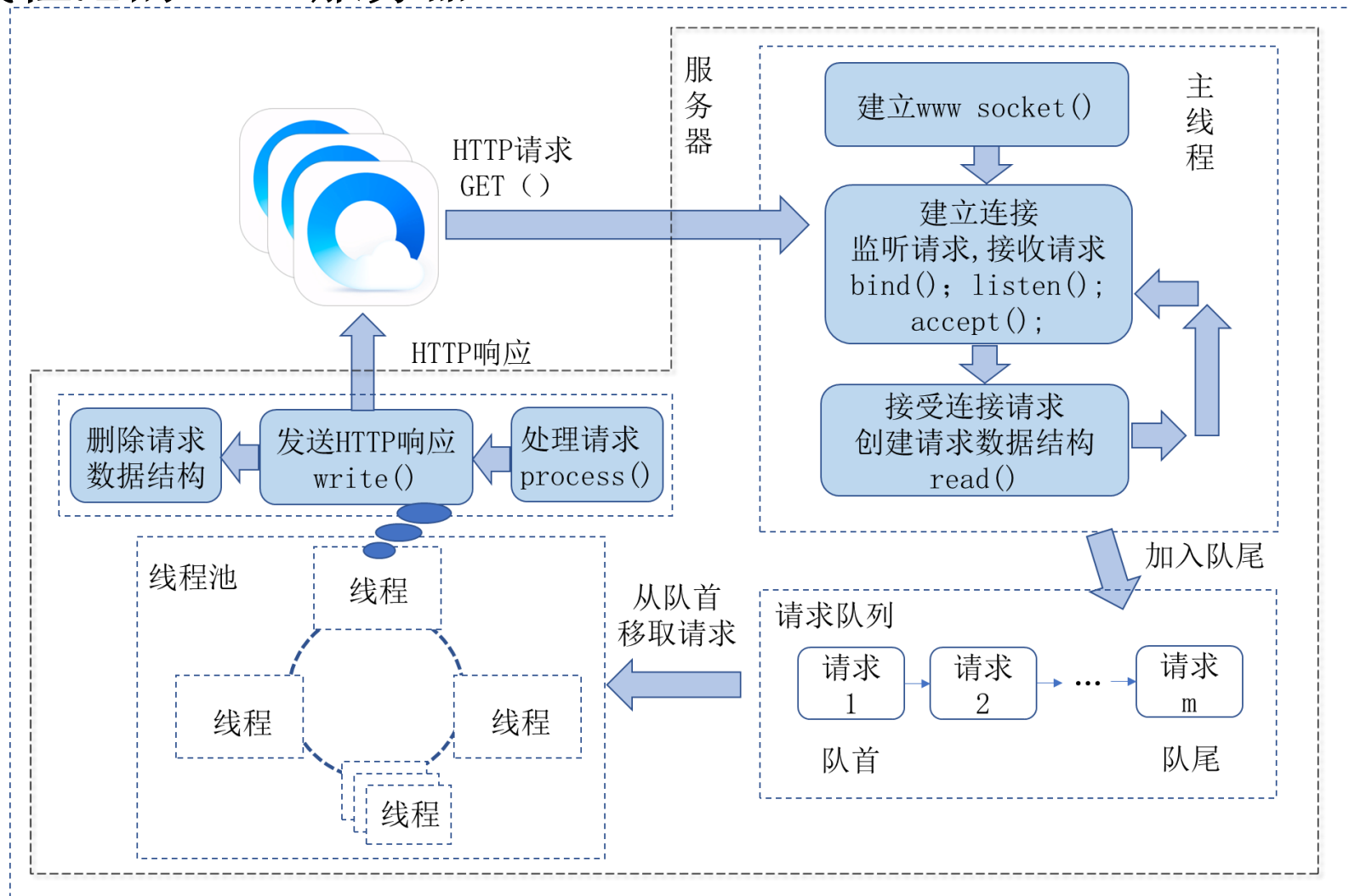
1) 选哪种实现方式?

2) 为什么?

3) 有没有更好的实现方式?

更好的方式

■ 基于线程池的Web 服务器



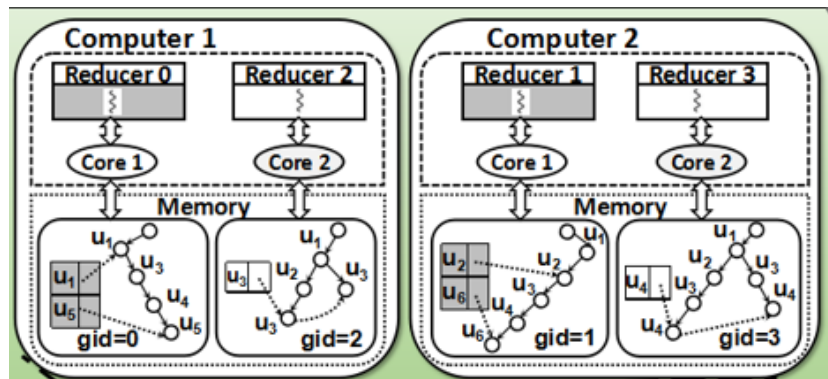
思考题：

以下哪种情况选用多进程比选用多线程更合适？答案：_____？

- A. Chrome浏览器的不同tabs；
- B. 当运行多个现有完整的程序；
- C. 当两个运行实体之间需要共享并交换信息时；
- D. Web服务器监听不同用户请求。

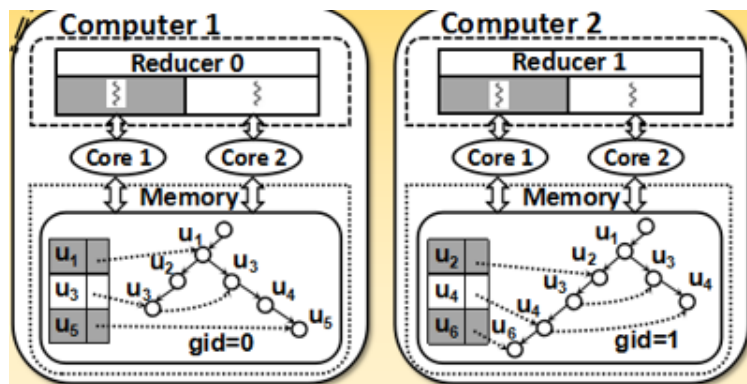


进程 vs. 线程：优化基于频繁模式树和多核Hadoop集群的计算



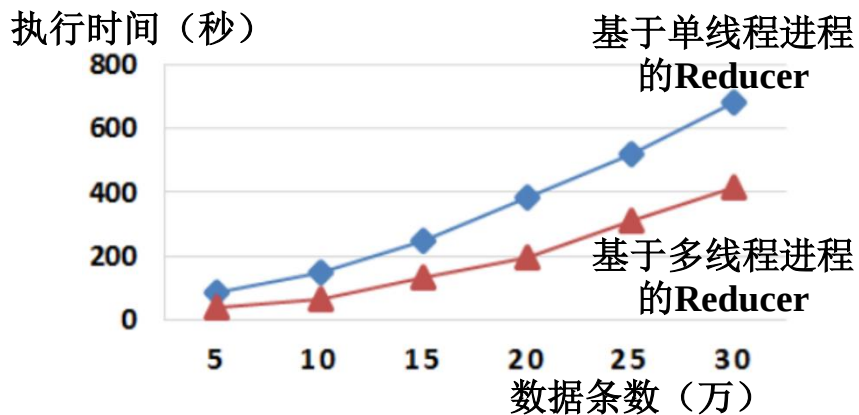
Reducer: 缺省，单线程进程

- × 内存占用高
- × 并发度低



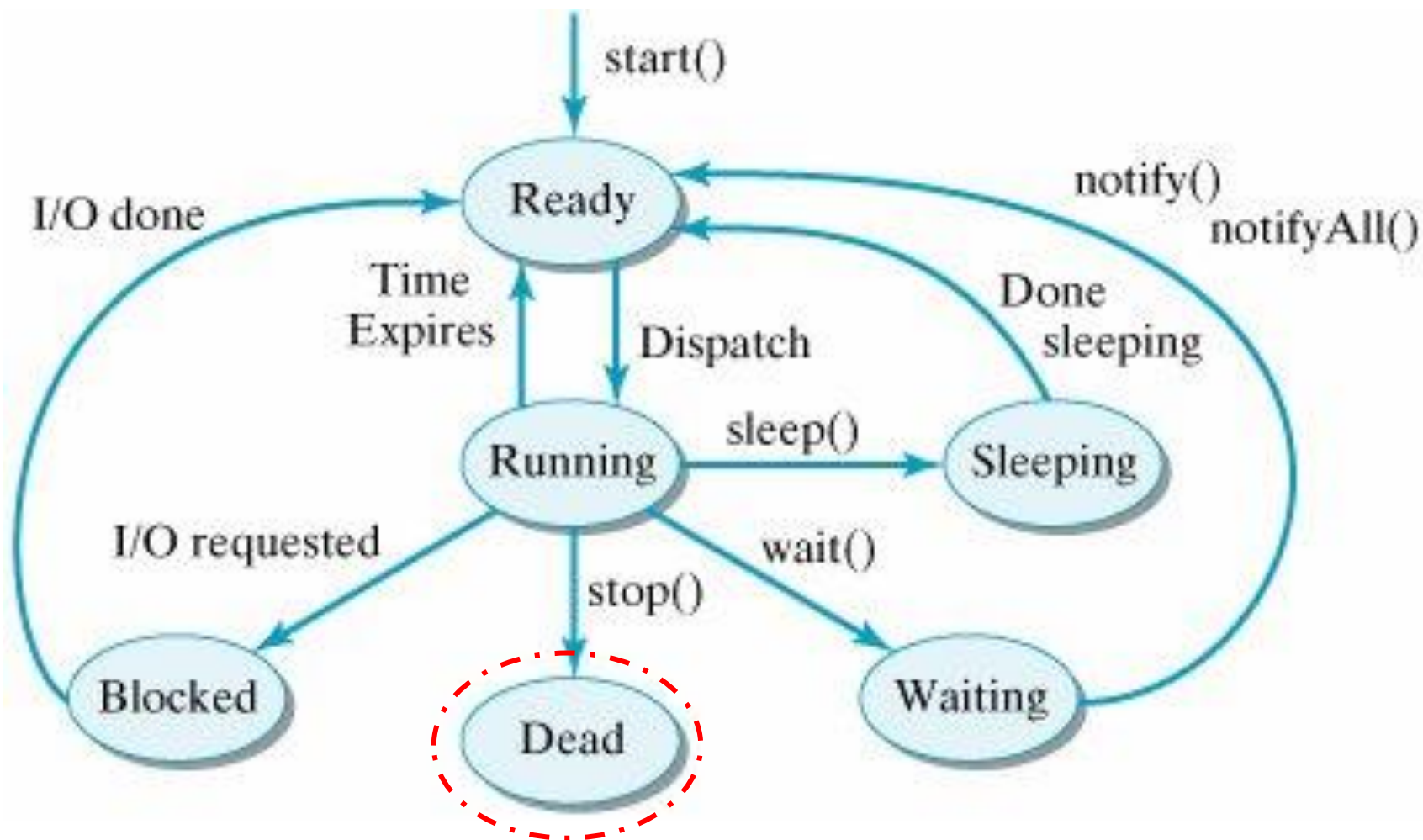
Reducer: 多线程，充分利用多核

- ✓ 内存占用少
- ✓ 并发度高

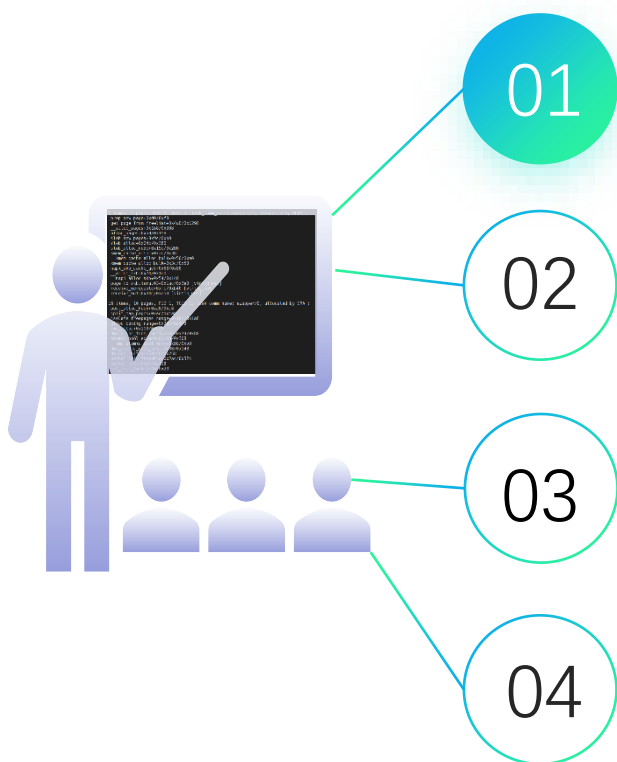


多线程技术可加速多核环境中的计算

线程状态



线程控制 – ② 系统调用



01

线程创建: `pthread_create()`

02

线程同步: `pthread_join()`

03

线程终止: `pthread_exit()/`
`pthread_cancel()`

04

小结: 基于线程的性能优化案例

线程控制 – ② 系统调用 – 线程创建

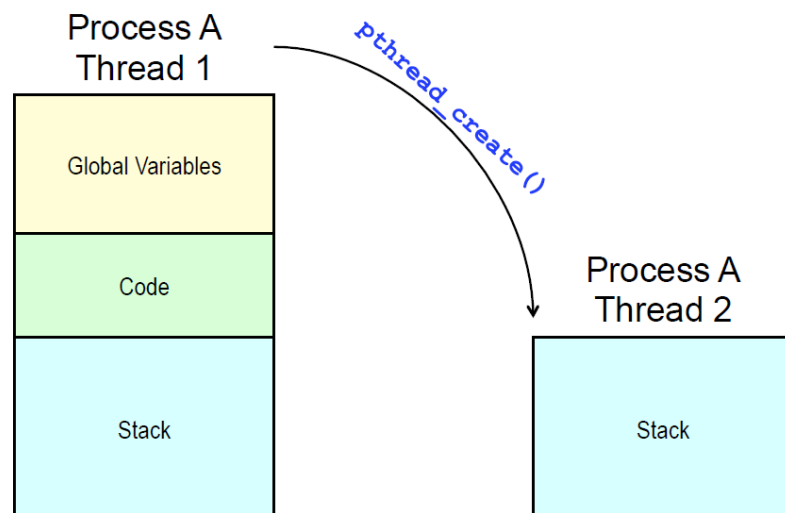
■ pthread_create()

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
                  void *(*start_routine) (void *), void *arg);
```

Compile and link with `-pthread`.

- thread: 指向线程标识符的指针
- attr: 设置线程属性
- start_routine: 线程函数的起址
- arg: 运行函数的参数



✓ 没有拷贝整个进程空间

线程控制 – ② 系统调用 – 线程创建

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
                  void *(*start_routine) (void *), void *arg);
```

```
#include <pthread.h>  
#include <stdio.h>  
void* print_char(void* unused)  
{  
    while (1) fputc(*((char *)unused),stderr);  
    return NULL;  
}  
int main(int argc, char *argv[])  
{  
    pthread_t tid1,tid2;  
    char c1 = '2', c2='3';  
    pthread_create(&tid1, NULL, &print_char, (void *)&c1);  
    pthread_create(&tid2, NULL, &print_char,(void *)&c2);  
    while (1) fputc('1',stderr);  
    return 0;  
}
```

```
[taishan02-vm-10 ~]$ ps axj | grep "threethreads"
```

842444	842630	842630	842444	pts/0	842444	Sl	1000	7:06	./threethreads
842674	842873	842872	842674	pts/1	842872	S+	1000	0:00	grep --color=au

```
to threethreads
```

```
[taishan02-vm-10 ~]$ pstree -p 842630
```

```
threethreads(842630)─{threethreads}(842643)  
                   └{threethreads}(842644)
```

```
[taishan02-vm-10 ~]$
```

线程控制 – ② 系统调用 – 线程创建

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
                  void *(*start_routine) (void *), void *arg);
```

■ void* 型参数：如果需要传输多项类型不同的数据

- 定义结构体类型, 包含所需的所有数据

- 多个线程可复用同一线程函数

- ◆ SIMD

```
#include <pthread.h>
```

```
pthread_t pthread_self(void);
```

■ 返回值：调用线程的ID, 永远成功

线程控制 – ② 系统调用 – 线程创建

```

#include <pthread.h>
#include <stdio.h>

struct n_chars {
    char chars;
    int  amount;
};

void *print_n_char(void *params)
{
    struct n_chars *p = (struct n_chars*)params;
    int i;

    for (i = 0; i < p->amount; i++)
        fputc(p->chars, stderr);
    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t t_id1, t_id2;
    struct n_chars t1_args, t2_args;

    t1_args.chars = '*';
    t1_args.amount = 3;
    pthread_create(&t_id1, NULL, &print_n_char, &t1_args);

    t2_args.chars = '^';
    t2_args.amount = 6;
    pthread_create(&t_id2, NULL, &print_n_char, &t2_args);
    return 0;
}
```

线程控制 – ② 系统调用 – 线程创建

```
#include <pthread.h>
#include <stdio.h>

struct n_chars {
    char chars;
    int  amount;
};

void *print_n_char(void *params)
{
    struct n_chars *p = (struct n_chars*)params;
    int i;

    for (i = 0; i < p->amount; i++)
        fputc(p->chars, stderr);
    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t t_id1, t_id2;
    struct n_chars t1_args, t2_args;

    t1_args.chars = '*';
    t1_args.amount = 3;
    pthread_create(&t_id1, NULL, &print_n_char, &t1_args);

    t2_args.chars = '^';
    t2_args.amount = 6;
    pthread_create(&t_id2, NULL, &print_n_char, &t2_args);
    return 0;
}
```

```
[root@taishan02-vm-10 threads]$ gcc -o join join.c -lpthread
[root@taishan02-vm-10 threads]$ ./join
***^
[root@taishan02-vm-10 threads]$ ./join
***^
[root@taishan02-vm-10 threads]$ ./join
***^
[root@taishan02-vm-10 threads]$ ./join
***^
[root@taishan02-vm-10 threads]$
```

- 创建进程的同时创建主线程
- 主线程管理进程资源
- 主线程退出
 - 结束进程
 - 回收资源
 - 终止它创建的线程(缺省)

线程控制 – ② 系统调用 – 线程同步

```
#include <pthread.h>
```

```
int pthread_join(pthread_t thread, void **retval);
```

Compile and link with -pthread.

■ 作用

- 等待指定线程的结束

■ 参数

- thread: 等待线程的ID
- retval: 等待线程函数的返回值

■ 返回值

- 成功: 0
- 出错: error number

出错代码	出错原因
EDEADLK	死锁
EINVAL	线程thread属性不是joinable 线程thread已被另一线程join
ESRCH	ID为thread的线程不存在

系统没有定义多线程等待同一线程的行为

2. 172.31.234.200

Re-attach Fullscreen Stay on top

```
#include <pthread.h>
#include <stdio.h>
```

```
struct n_chars {
    char chars;
    int amount;
```

```
};
```

```
void *print_n_char(void *params)
{
    struct n_chars *p = (struct n_chars*)params;
    int i;

    for (i = 0; i < p->amount; i++)
        fputc(p->chars, stderr);
    return NULL;
}
```

```
int main(int argc, char *argv[])
{
    pthread_t t_id1, t_id2;
    struct n_chars t1_args, t2_args;

    t1_args.chars = '*';
    t1_args.amount = 3;
    pthread_create(&t_id1, NULL, &print_n_char, &t1_args);

    t2_args.chars = '^';
    t2_args.amount = 6;
    pthread_create(&t_id2, NULL, &print_n_char, &t2_args);

    pthread_join(t_id1, NULL);
    pthread_join(t_id2, NULL);
    return 0;
}
```

```
@taishan02-vm-10 threads]$ gcc -o withjoin withjoin.c -lpthread
@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^
@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^
@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^
@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^
@taishan02-vm-10 threads]$ ./withjoin
***^^^^^^
@taishan02-vm-10 threads]$
```

■ 等待线程结束，释放相应资源

- 线程可选择自己释放资源

- ◆ 线程属性为detached

- ◆ 不能被其他线程调用
pthread_join() 同步

■ 等待joinable线程失败

- 僵死线程(zombie thread)

- 系统资源不足

线程控制 – ② 线程控制系统调用 – 线程终止

■ 自主终止

- 线程函数结束

- 函数 `pthread_exit()`

```
#include <pthread.h>
```

```
void pthread_exit(void *retval);
```

```
int pthread_join(pthread_t thread, void **retval);
```



If `retval` is not `NULL`, then `pthread_join()` copies the exit status of the target thread (i.e., the value that the target thread supplied to `pthread_exit(3)`) into the location pointed to by `*retval`. If the target thread was canceled, then `PTHREAD_CANCELED` is placed in `*retval`.

获取线程退出状态的例程（一）

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

void* adder(void* arg)
{
    int *operand = malloc(sizeof(int));
    *operand = *((int *)arg);
    *operand = 2 + *operand;

    pthread_exit((void*)operand);
}

int main()
{
    pthread_t thread;
    int *result;
    const int operand=3;
    pthread_create(&thread, NULL, &adder, (void *)&operand);
    pthread_join(thread, (void *)&result);
    printf("The result of the adder is %d.\n", *result);
}
```

```
@taishan02-vm-10 threads]$ gcc -lpthread -o returnvalue returnvalue.c
@taishan02-vm-10 threads]$ ./returnvalue
The result of the adder is 5.
```

```

@taishan02-vm-10 threads]$ gcc -lpthread -o exitstatus exitstatus.c
@taishan02-vm-10 threads]$ ./exitstatus
Thread 1 exits with code 1
Thread 2 exits with code 2
@taishan02-vm-10 threads]$ cat exitstatus.c
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

void *thr_fn1(void *arg)
{
    return ((void *)1);
}

void *thr_fn2(void *arg)
{
    pthread_exit((void *)2);
}

int main()
{
    pthread_t tid1, tid2;
    void *tret1, *tret2;

    if (pthread_create(&tid1, NULL, thr_fn1, NULL) < 0) {
        perror("Fail to create pthread tid1.\n");
        exit(-1);
    }

    if (pthread_create(&tid2, NULL, thr_fn2, NULL) < 0) {
        perror("Fail to create pthread tid2.\n");
        exit(-1);
    }

    pthread_join(tid1, &tret1);
    pthread_join(tid2, &tret2);
    printf("Thread 1 exits with code %d\n", (int *)tret1);
    printf("Thread 2 exits with code %d\n", (int *)tret2);
    exit(0);
}
@taishan02-vm-10 threads]$ █

```

获取线程退出状态的例程（二）

线程控制 – ② 线程控制系统调用 – 线程终止

■ 被动终止: `pthread_cancel()`

```
#include <pthread.h>
```

```
int pthread_cancel(pthread_t thread);
```

成功: 返回0 失败: 返回非0的出错代码

成功并不意味着thread会终止, 与目标线程的两个属性有关

■ cancelability state

- `PTHREAD_CANCEL_ENABLE` (缺省): 取决于cancelability type

- ◆ `PTHREAD_CANCEL_ASYNCIOUS`: 任意时间都可取消该线程

- ◆ `PTHREAD_CANCEL_DEFERRED`: 设置可取消点, 收到取消请求, 将其放进队列, 等线程运行至下一个取消点再退出

- `PTHREAD_CANCEL_DISABLE`: 取消请求被阻塞直到状态改变

```

#include <pthread.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>
#define handle_error_en(en, msg) \
    do { errno = en; perror(msg); exit(EXIT_FAILURE); } while (0)

static void * thread_func(void *ignored_argument)
{
    int s;
    if ((s = pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL)) != 0)
        handle_error_en(s, "pthread_setcancelstate");
    printf("thread_func(): started; cancellation disabled\n");
    sleep(5);
    printf("thread_func(): about to enable cancellation\n");
    if ((s = pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL)) != 0)
        handle_error_en(s, "pthread_setcancelstate");
    sleep(1000); /* Should get canceled while we sleep */
    /* Should never get here */
    printf("thread_func(): not canceled!\n");
    return NULL;
}

```

```

int main(void)
{
    pthread_t thr;
    void *res;
    int s;
    if ((s = pthread_create(&thr, NULL, &thread_func, NULL)) != 0)
        handle_error_en(s, "pthread_create");
    sleep(2); /* Give thread a chance to get started */
    printf("main(): sending cancellation request\n");
    if ((s = pthread_cancel(thr)) != 0)
        handle_error_en(s, "pthread_cancel");
    if ((s = pthread_join(thr, &res)) != 0)
        handle_error_en(s, "pthread_join");
    if (res == PTHREAD_CANCELED)
        printf("main(): thread was canceled\n");
    else
        printf("main(): thread wasn't canceled (shouldn't happen!)\n");
    exit(EXIT_SUCCESS);
}

```

```

[taishan02-vm-10 threads]$ gcc -o cancelthread cancelthread.c -lpthread
[taishan02-vm-10 threads]$ ./cancelthread
thread_func(): started; cancellation disabled
main(): sending cancellation request
thread_func(): about to enable cancellation
main(): thread was canceled

```

■ pthread_cancel()
应用例子

■ 在man手册页中